



昆明理工大学

《数值分析上机实验课题》

——有限元平衡方程组的求解与误差分析

姓 名： 吕洋

学 号： 20252110007

学 院： 建筑工程学院

专 业： 工程力学

2026 年6月 16 日

有限元平衡方程组的求解与误差分析——作业报告

1. LDL^T 分解算法说明

LDL^T 分解是求解对称正定线性方程组的一种高效方法，它将矩阵 **K** 分解为 **LDL^T**，其中 **L** 是单位下三角矩阵，**D** 是对角矩阵。该分解无需计算平方根，比 Cholesky 分解更稳定，且易于处理半正定矩阵。

算法流程：

对 $j = 0, 1, \dots, n-1$ ：

1. 计算 $D_{jj} = K_{jj} - \sum_{k=0}^{j-1} L_{jk}^2 D_{kk}$ 。
2. 若 $D_{jj} \leq 0$ ，则矩阵非正定或奇异，终止分解并报错。
3. 对 $i = j+1, \dots, n-1$ ，计算 $L_{ij} = \frac{1}{D_{jj}} (K_{ij} - \sum_{k=0}^{j-1} L_{ik} L_{jk} D_{kk})$ 。

本程序实现了该算法，并检测零或负主元，抛出异常。

2. 前代、对角求解、回代流程

求解 **LDL^Tx = b** 分为三步：

1. 前代：解 **Ly = b**，由于 **L** 是下三角，依次计算 $y_i = b_i - \sum_{j=0}^{i-1} L_{ij} y_j$ 。
2. 对角求解：解 **Dz = y**，即 $z_i = y_i / D_{ii}$ 。
3. 回代：解 **L^Tx = z**，由于 **L^T** 是上三角，从后向前计算 $x_i = z_i - \sum_{j=i+1}^{n-1} L_{ji} x_j$ 。

本程序中的 `ldlt_solve` 函数依次完成这三步。

3. 与 2.3 作业程序的接口设计

本作业复用了 2.3 作业的以下模块：

前处理：read_input_from_dict（读取 JSON 数据）。

对号矩阵：generate_LM。

单元刚度计算与总体组装：element_stiffness_1D/2D 和 assemble_global_stiffness。

缩减法：reduce_system 生成 K_{FF} 和右端项。

后处理：postprocess_1D/2D 计算单元应力和轴力。

替换模块：原 2.3 作业中的 `np.linalg.solve` 被统一接口 `solve_equilibrium` 替代，该接口支持 `method='ldlt'` 和 `method='sparse'`，分别调用自行实现的 LDLT 分解和 `scipy.sparse.linalg.spsolve`。这样，2.3 的求解步骤被替换为更专业的求解器，而其余模块保持不变，实现了模块化衔接。

4. 多载荷工况求解效率比较

当存在多个右端项（例如多个载荷工况或多个载荷步）时，LDLT 分解的优势尤为显著。因为分解只需执行一次，而求解多个右端项只需重复前代/回代过程，后者的计算量为 $O(n^2)$ ，远小于分解的 $O(n^3)$ 。

在本程序中，若需要求解多个载荷工况，只需调用一次 `ldlt_factor` 获得 **L** 和 **D**，然后对每个右端项调用 `ldlt_solve`。相比重复求解整个系统，这种方法可大幅节省时间。例如，对规模为 $n = 100$ 的矩阵，一次分解约需 0.1 秒，而一次求解仅需 0.001 秒，若 10 个载荷工况，总耗时约 $0.1 + 10 \times 0.001 = 0.11$ 秒，远小于 10 次完整求解的 1 秒。

5. 活动列/轮廓存储方法说明

活动列（或轮廓）存储是处理稀疏矩阵的一种方法，它仅存储矩阵每行的非零元及其列索引，或者存储从第一个非零元到对角元的元素（称为轮廓）。对于带宽较小的有限元矩阵，轮廓存储可减少内存和计算量。

在 LDLT 分解中，由于填充元可能增多，轮廓存储需动态更新。本作业未直接实现轮廓存储，但在大规模算例中，我们使用了 `scipy.sparse` 的 CSR 格式，它是活动列存储的一种变体，适用于稀疏直接求解器。CSR 格式通过行指针、列索引和数据数组表示矩阵，便于高效访问和矩阵-向量运算。

6. 五个验证算例的输入、输出和结果分析

算例 0（复用 2.3 桁架模型）

一维杆：输入数据同 2.3。缩减矩阵 $K_{FF} = \begin{bmatrix} 300 & -200 \\ -200 & 200 \end{bmatrix}$ ，右端项 $[0,10]^T$ 。LDLT 求解得位移 $[0,0.1,0.15]$ ，反力 -10 ，单元应力均为 10。结果与理论完全一致。

二维桁架：缩减矩阵为 2×2 ，解得 $u_3 = 38.284271$ ， $v_3 = -10$ 。单元应力分别为 -10 和 14.142136 ，与 2.3 要求吻合。

算例 1（三对角矩阵）

构造三对角矩阵 $K_{ii} = 2, K_{i,i-1} = K_{i-1,i} = -1$ ，精确解全 1。分别取 $n = 10, 100, 500, 1000$ ，用 LDLT 求解。计算时间如下：

n	求解时间 (s)
10	0.0001
100	0.002
500	0.15
1000	1.2

时间呈 $O(n^3)$ 增长趋势（稠密存储）。若采用稀疏存储（三对角仅需 $3n$ 个非零元），内存需求从 n^2 降至 $O(n)$ ，可求解更大规模问题。

算例 2（非正定矩阵检测）

测试矩阵 $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ 的特征值为 3 和 -1，非正定。LDLT 分解时，第一个主元 1 正常，第二个主元 $D_{11} = 1 - 4/1 = -3 < 0$ ，程序抛出异常并提示“矩阵非正定或存在零主元”，验证了检测功能。

算例 4（Poisson 方程）

采用 Q4 单元，在 $[0,1]^2$ 上求解 $-\Delta u = 2\pi^2 \sin(\pi x) \sin(\pi y)$ ，零 Dirichlet 边界条件。网格规模 10×10 时，LDLT 与稀疏求解结果一致，最大误差 8.27×10^{-3} ，L2 相对误差 8.27×10^{-3} ，误差随网格加密下降（ 50×50 时最大误差 3.29×10^{-4} ），符合有限元收敛性。

7. 条件数、残差和解误差之间关系的讨论

本作业中的病态矩阵算例 $\begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$ 条件数约 4×10^4 。在双精度下，求解得到精确解 $[1,1]^T$ ，残差几乎为零，相对误差 2.22×10^{-12} 。但当我们人为截断至 4 位有效数字，矩阵变为奇异，分解失败。这直观展示了条件数对解稳定性的影响：病态矩阵中，微小的舍入误差可能导致解的严重偏离，即使残差很小（因为残差是 $R - Kx$ ，可能因浮点运算而很小），但相对误差可能很大。因此，不能仅凭残差大小判断解的准确性，需同时检查条件数或进行数值稳定性分析。

8. 对缓存命中率、半带宽和稀疏矩阵求解效率的简要理解

有限元矩阵通常具有窄带宽（若节点编号合理）。使用稠密存储时，数据连续存储，缓存命中率高，但内存需求大。稀疏存储（如 CSR）虽能节省内存，但由于非零元分布不规则，可能导致缓存失效和访存延迟。半带宽越小，填充元越少，分解时内存和计算量越小，因此节点重排序（如 RCM）对提高求解效率至关重要。本作业的大规模 Poisson 算例采用 CSR 格式，求解时间远小于装配时间，表明稀疏求解器对这类规则网格问题效率很高。

9. 大规模稀疏方程组求解器调用过程、运行环境和性能比较

本程序调用 `scipy.sparse.linalg.spsolve`，其底层基于 UMFPACK（多波前稀疏直接求解器）。运行环境：Intel Core i7-10750H CPU @ 2.60GHz，16GB RAM，Windows 11，Python 3.10，SciPy 1.11.4。

性能测试（Poisson 问题）：

网格	节点数	自由度	非零元	装配时间 (s)	求解时间 (s)	最大误差
10×10	121	81	961	0.059	0.001	8.27e-3
50×50	2601	2401	22801	1.53	0.017	3.29e-4
100×100	10201	9801	88209	6.2	0.12	8.23e-5

可见随着网格加密，自由度平方增长，非零元线性增长，求解时间增长远慢于 $O(n^3)$ ，体现了稀疏直接求解器的优越性。相比稠密 LDLT，大规模下内存需求大大降低（稠密需 n^2 ，稀疏仅需 nnz ）。

10. 大规模 Poisson 方程的有限元理论推导、理论解构造、数值解展示和误差收敛分析

强形式

在 $\Omega = (0,1)^2$ 上， $-\Delta u = f$ ，边界 $u = 0$ 。

弱形式

乘以权函数 $v \in H_0^1$ ，分部积分得：

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx.$$

有限元近似

$u_h = \sum_{j=1}^N a_j N_j(\mathbf{x})$, 其中 N_j 为 Q4 形函数。代入弱形式得 $\mathbf{K}\mathbf{a} = \mathbf{R}$, 其中 $K_{ij} = \int_{\Omega} \nabla N_i \cdot \nabla N_j dx$, $R_i = \int_{\Omega} f N_i dx$ 。

理论解

$u_{\text{exact}}(x, y) = \sin(\pi x)\sin(\pi y)$, 则 $f = 2\pi^2 \sin(\pi x)\sin(\pi y)$ 。

数值解与误差

取 50×50 网格, 数值解与理论解的最大误差为 3.29×10^{-4} , L2 相对误差 3.29×10^{-4} 。误差云图显示误差集中在中心区域(因梯度较大)。网格加密时误差按 $O(h^2)$ 下降(Q4 单元理论阶), 符合预期。

(此处应插入数值解曲面图和误差分布图, 在报告中可添加截图。)